Practical 09 --- Describing the Service to a Clu

Deployment, Service and probes, written as YAML and checked before they run

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L15-L17
Course Outcome	CO4
Lab duration	120 minutes
Memory needed	about 200 MB (fine on a 4 GB or 8 GB machine)
Extra software	nothing beyond the course venv
Marks	10

Aim

1. Explain what a Deployment and a Service each do, in plain words.
2. Write both as YAML manifests for the delivery-time container.
3. Check a manifest is correct before it ever reaches a cluster.
4. Add readiness and liveness probes, and say what each one is for.

What you need to know first

In P08 you ran one container with `docker run`. That works until you need a second copy, or t
machine reboots, or the container dies at two in the morning. Somebody has to notice and start i

Kubernetes is the software that does the noticing. You do not tell it what to do; you tell it **what y**
and it works continuously to make reality match. That style is called **declarative**, and it is the sin
behind everything today.

Three objects matter for us.

A **Pod** is one running copy of your container. It is the smallest thing Kubernetes handles. Pods a
disposable: they get killed and replaced, and they get a new address every time.

A **Deployment** is a standing order: *keep three copies of this image running at all times*. If one di
Deployment makes another. If you change the image, it replaces them one at a time so the servi
goes down.

A **Service** is one permanent address in front of those changing Pods. Since Pods come and go
addresses, nothing could reliably call them. The Service is the fixed name that spreads requests
whichever Pods exist now.

The glue between Deployment and Service is **labels**. A Deployment stamps a label on its Pods; Service says "send traffic to anything with this label". If those two strings do not match, everythin appears healthy and nothing works. That is the most common Kubernetes mistake in the world, will fix it in Task T3.

Finally, **probes**. Kubernetes cannot guess what "working" means for your service, so you tell it. **readiness** probe asks *should traffic come yet?* --- a model that takes 30 seconds to load should receive requests at second 2. A **liveness** probe asks *are you still alive?* --- if it fails, the containe restarted. Confusing the two is how one slow pod becomes an endless restart loop.

```
---

## Before you start

- Practical P08 is finished. You have an image called `delivery-
api:1.0`.
```

- You have seen YAML in P05. Remember: spaces only, never tabs.
- `kubectl` and a cluster are **optional** today. Every graded task is about writing correct mar which you can do on any machine. If Docker Desktop has Kubernetes enabled, the noteboo also run the real commands for you.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to th one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier created. If you jump ahead you will see a `NameError`, which just means "you have not ma thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, start again from the first cell. Nothing is damaged by doing this.

In [ ]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['yaml']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)
```

```
for _name in ['yaml']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Check whether a cluster is available

This is optional today. If `kubectl` is installed the notebook will run real commands for you; if n
everything else still works, because writing correct manifests is the actual skill.

In [ ]:
```
import shutil
import subprocess
from pathlib import Path

import yaml

WORK = Path("work")
WORK.mkdir(exist_ok=True)

KUBECTL_OK = shutil.which("kubectl") is not None
CLUSTER_OK = False

if KUBECTL_OK:
    probe = subprocess.run(
        ["kubectl", "cluster-info"], capture_output=True,
        text=True, timeout=30,
    )
    CLUSTER_OK = probe.returncode == 0
    print("kubectl found. Cluster reachable:", CLUSTER_OK)
else:
    print("kubectl is not installed on this machine.")
    print("That is fine. Every task today is about writing correct")
    print("YAML, and the notebook checks your YAML itself.")

print()
print("KUBECTL_OK =", KUBECTL_OK, "  CLUSTER_OK =", CLUSTER_OK)


def kubectl(*args):
    """Run one kubectl command, or explain why it was skipped."""
    if not CLUSTER_OK:
        print(f"SKIPPED (no cluster): kubectl {' '.join(args)}")
        return None
```

```
    print(f"$ kubectl {' '.join(args)}")
    done = subprocess.run(["kubectl", *args], capture_output=True,
                          text=True, timeout=120)
    print((done.stdout + done.stderr).strip()[-2000:])
    print("-" * 60)
    return done
```

Step 2 --- Write the Deployment

Read it top to bottom. Every manifest has the same four opening keys: `apiVersion`, `kind`, `metadata`, `spec`.

The part that catches everybody: there are **two** `metadata` blocks. The outer one names the Deployment. The inner one, under `template`, names the Pods it will create. The `selector` middle says which Pods this Deployment is responsible for --- and it must match the template's l exactly.

In [ ]:
```
deployment_yaml = """apiVersion: apps/v1
kind: Deployment
metadata:
  name: delivery-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: delivery-api
  template:
    metadata:
      labels:
        app: delivery-api
    spec:
      containers:
        - name: api
          image: delivery-api:1.0
          ports:
            - containerPort: 8000
          resources:
            requests:
              memory: "256Mi"
              cpu: "100m"
            limits:
              memory: "512Mi"
              cpu: "500m"
"""

(WORK / "deployment.yaml").write_text(deployment_yaml,
                                      encoding="utf-8")
print(deployment_yaml)
```

`replicas: 3` is the standing order. `resources` tells the cluster how much this container ne it can decide which machine has room --- and stops one runaway pod from eating a whole node.

Step 3 --- Check the YAML before anybody runs it

A manifest is just text. The fastest way to catch a mistake is to load it in Python and look at it --- r
cluster needed.

This habit is worth more than it looks: in P11 a pipeline will do exactly this check automatically, b
deploying.

In [ ]:
```
with open(WORK / "deployment.yaml", encoding="utf-8") as fh:
    deployment = yaml.safe_load(fh)

print("kind      :", deployment["kind"])
print("name      :", deployment["metadata"]["name"])
print("replicas  :", deployment["spec"]["replicas"])

selector = deployment["spec"]["selector"]["matchLabels"]
pod_labels = deployment["spec"]["template"]["metadata"]["labels"]

print("selector  :", selector)
print("pod labels:", pod_labels)
print()
print("labels match:", selector == pod_labels,
      "  <- if this is False, nothing will work")
```

Step 4 --- Write the Service

The Service is much shorter. Two things matter.

`selector` must match the Pod labels from the Deployment --- that is how it finds them.

`port` is the address the Service answers on; `targetPort` is the port inside the container. Tl
allowed to differ, and confusing them is the second most common mistake.

In [ ]:
```
service_yaml = """apiVersion: v1
kind: Service
metadata:
  name: delivery-api-svc
spec:
  type: ClusterIP
  selector:
    app: delivery-api
  ports:
    - port: 80
      targetPort: 8000
"""

(WORK / "service.yaml").write_text(service_yaml, encoding="utf-8")
print(service_yaml)
```

`ClusterIP` means "reachable inside the cluster only". That is the right default. `NodePort` a
`LoadBalancer` open it up further, and you should only reach for them when you actually neec

Step 5 --- Prove the Service can find the Pods

Load both files and compare the two strings that must agree. This three-line check would have s great many careers' worth of afternoons.

In [ ]:
```
with open(WORK / "service.yaml", encoding="utf-8") as fh:
    service = yaml.safe_load(fh)

svc_selector = service["spec"]["selector"]

print("Service selects :", svc_selector)
print("Pods are labelled:", pod_labels)
print()
if svc_selector == pod_labels:
    print("MATCH. Traffic will reach the pods.")
else:
    print("MISMATCH. The Service would find nothing,")
    print("and Kubernetes would not warn you.")
```

Step 6 --- Add the health probes

Now teach the cluster what healthy means. Both probes call the `/health` endpoint you wrote

Look at the two `initialDelaySeconds` values and think about why they differ. Readiness st asking early, because we want traffic as soon as possible. Liveness waits much longer, because container that is merely still starting up is a disaster.

In [ ]:
```
probes = {
    "readinessProbe": {
        "httpGet": {"path": "/health", "port": 8000},
        "initialDelaySeconds": 5,
        "periodSeconds": 10,
    },
    "livenessProbe": {
        "httpGet": {"path": "/health", "port": 8000},
        "initialDelaySeconds": 30,
        "periodSeconds": 20,
        "failureThreshold": 3,
    },
}

container = deployment["spec"]["template"]["spec"]["containers"][0]
container.update(probes)

(WORK / "deployment.yaml").write_text(
    yaml.safe_dump(deployment, sort_keys=False), encoding="utf-8"
)

print(yaml.safe_dump(container, sort_keys=False))
```

We edited the manifest as a Python dictionary and wrote it back. That is a genuinely useful trick, how a pipeline changes an image tag automatically in P11.

Step 7 --- What each probe does when it fails

Say this out loud until it sticks.

In [ ]:
```
print("readiness fails -> the pod is taken OUT of the Service")
print("                   no traffic arrives; the pod keeps running")
print()
print("liveness  fails -> the container is KILLED and restarted")
print()
print("So:")
print("  slow to start        -> raise readiness initialDelaySeconds")
print("  hung, needs a kick   -> that is what liveness is for")
print("  only a liveness probe-> traffic hits a pod that is not ready")
```

Step 8 --- Send it to a cluster, if you have one

These are the real commands. They run here only if a cluster is reachable; otherwise you will se printed, which is what you need for your notes.

`apply` is declarative: run it twice and nothing bad happens. It makes reality match the file.

In [ ]:
```
kubectl("apply", "-f", str(WORK / "deployment.yaml"))
kubectl("apply", "-f", str(WORK / "service.yaml"))
kubectl("get", "deployment", "delivery-api")
kubectl("get", "pods", "-l", "app=delivery-api")
kubectl("get", "service", "delivery-api-svc")

print()
print("The commands above, for your notes:")
for line in [
    "kubectl apply -f deployment.yaml",
    "kubectl apply -f service.yaml",
    "kubectl get pods -l app=delivery-api",
    "kubectl describe pod <pod-name>      # read Events at the end",
    "kubectl logs <pod-name>",
    "kubectl rollout status deployment/delivery-api",
    "kubectl rollout undo deployment/delivery-api",
    "kubectl delete -f deployment.yaml -f service.yaml",
]:
    print("   ", line)
```

Step 9 --- Reading a broken deployment

You will meet these four states constantly. Learn them now; it saves hours later.

In [ ]:
```
states = [
    ("Pending", "no machine has room, or a volume is missing"),
    ("ContainerCreating", "pulling the image; normal for a minute"),
    ("ImagePullBackOff", "wrong image name or tag, or no access"),
    ("CrashLoopBackOff", "it starts, dies, restarts, forever"),
    ("Running", "the process started -- which is not the same as working"
]

for state, meaning in states:
    print(f"  {state:<20} {meaning}")
```

```
print()
print("First command every time: kubectl describe pod <name>")
print("Then read the Events list at the very bottom. The reason")
print("is almost always there.")
```

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you whic are correct. You can run it as many times as you like.

Task T1 --- Write a Service for a second version

```
Write `work/service-v2.yaml`: a Service named `delivery-api-v2-svc`
that:

- is of type `NodePort`
- selects Pods labelled `app: delivery-api-v2`
- answers on port `80` and forwards to `targetPort` `8000`

Load it back with `yaml.safe_load` and store the dictionary in
`T1_service`.
```

> **Hint.** Copy the Service from Step 4 and change four things: the name, the type, the selector value and nothing else. Write it with `write_text`, then read it back with `yaml.safe_load` so the check can see it.

In [ ]:
```
service_v2 = """apiVersion: v1
kind: Service
metadata:
  name: TODO
spec:
  type: TODO
  selector:
    app: TODO
  ports:
    - port: 0
      targetPort: 0
"""

(WORK / "service-v2.yaml").write_text(service_v2, encoding="utf-8")

# TODO: load the file back into a dictionary
T1_service = None
```

```
print(T1_service)
```

Task T2 --- Probes for a slow model

A new model takes **40 seconds** to load. The Deployment in Step 6 a
liveness probe with `initialDelaySeconds: 30`, which would kill it
mid-load, forever.

Build a dictionary `T2_probes` with two keys, `readinessProbe` and
`livenessProbe`, both calling `/health` on port 8000, where:

- readiness starts after **10** seconds
- liveness starts after **60** seconds --- comfortably past the 40
  needs

> **Hint.** Copy the `probes` dictionary from Step 6 and change the two `initialDelaySeconds` values. Keep the `httpGet` blocks exactly as they are.

In [ ]:
```
# TODO: two probes, both httpGet /health on port 8000,
#       readiness after 10 seconds, liveness after 60.
T2_probes = None

print(T2_probes)
```

Task T3 --- Find the mismatch

The two manifests below look fine and would deploy without a single
warning --- and no traffic would ever reach a pod.

Load them both, find the fault, and fix it by editing the **Service**
so that it selects the Pods the Deployment actually creates.

Put the corrected Service dictionary in `T3_fixed_service`.

> **Hint.** Print `broken_deploy['spec']['template']['metadata']['labels']` and `broken_svc['spec']['selector']` and read them side by side. One word differs. Change the Service's selector to match the Pod labels exactly.

In [ ]:
```
broken_deploy = yaml.safe_load("""
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payments-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: payments-api
  template:
    metadata:
      labels:
```

```
        app: payments-api
    spec:
      containers:
        - name: api
          image: payments-api:3.1
""")

broken_svc = yaml.safe_load("""
apiVersion: v1
kind: Service
metadata:
  name: payments-svc
spec:
  selector:
    app: payment-api
  ports:
    - port: 80
      targetPort: 8080
""")

# TODO: fix the Service so its selector matches the pod labels
T3_fixed_service = None

print(T3_fixed_service)
```

Self-check

Run the cell below to mark your work.

In [ ]:

```
# ----------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ----------------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | it is a Service with the right name', lambda: T1_service['ki
_check('T1 | its type is NodePort and it selects delivery-api-v2', lambda
_check('T1 | port 80 forwards to targetPort 8000', lambda: T1_service['sp
_check('T2 | both probes are present', lambda: set(T2_probes) == {'readin
_check('T2 | both call /health on port 8000', lambda: all(p['httpGet'] ==
_check('T2 | readiness starts at 10s and liveness at 60s', lambda: T2_pro
_check('T2 | liveness waits longer than readiness', lambda: T2_probes['li
_check('T3 | the Service selector now matches the pod labels', lambda: T3
_check('T3 | the selector is app: payments-api, spelled correctly', lambd
```

```
_check('T3 | nothing else about the Service was damaged', lambda: T3_fixe

print("======================================================================
print("SELF-CHECK   Practical 09 --- Describing the Service to a Cluster"
print("======================================================================
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------------
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("======================================================================
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell agai
```

What to submit

1. This notebook, with every cell run and its output visible.
2. Your `work/deployment.yaml` and `work/service.yaml`.
3. In a markdown cell, two sentences: what happens when a readiness probe fails, and what h when a liveness probe fails.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, manifests written and checked	3
Task T1 --- a correct Service manifest	2
Task T2 --- probes added with sensible delays	3
Task T3 --- the label mismatch found and fixed	2
Total	**10**

Read more

- Kubernetes --- Deployments --- https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Kubernetes --- Service --- https://kubernetes.io/docs/concepts/services-networking/service/
- Kubernetes --- Configure liveness, readiness and startup probes --- https://kubernetes.io/do configure-pod-container/configure-liveness-readiness-startup-probes/
- Kubernetes --- kubectl quick reference --- https://kubernetes.io/docs/reference/kubectl/quick reference/